

Pesquisa profunda: Git, versionamento de código, GitHub, GitLab e plataformas modernas

Conteúdo formatado em PDF técnico, preservando a estrutura original do material anexado.

Tema central	Git, versionamento de código, GitHub, GitLab e plataformas modernas
Nível	Do básico ao avançado, com foco em maturidade sênior
Fonte	Texto colado(70).txt
Versão	Edição formatada e padronizada

Material organizado como apostila de estudo, com blocos de comando, exemplos, tabelas e seções hierárquicas para facilitar revisão e consulta.

Sumário

Estrutura navegável das principais seções do documento.

1. Visão geral	4
2. Objetivo da trilha	4
3. Trilha progressiva por nível	4
Nível básico — Fundamentos essenciais	4
3.1 O que é controle de versão	4
3.2 Estados principais do Git	5
3.3 Comandos básicos obrigatórios	5
4. Nível intermediário — Trabalho em equipe e fluxo real	5
4.1 Branches	5
4.2 Merge vs Rebase	6
4.3 Conflitos de merge	6
4.4 Pull Request / Merge Request	7
5. Nível avançado — Git interno, qualidade e automação	7
5.1 Modelo interno do Git	7
5.2 HEAD, refs e histórico	7
5.3 Reset, revert e restore	8
5.4 Stash	8
5.5 Cherry-pick	8
5.6 Bisect	9
5.7 Hooks	9
6. Convenções profissionais de commits	9
6.1 Conventional Commits	9
6.2 SemVer	10
7. Estratégias de branching	10
7.1 GitHub Flow	10
7.2 GitFlow	10
7.3 Trunk-Based Development	11
7.4 Release branches	11
8. Plataformas: GitHub, GitLab, Bitbucket, Azure Repos e alternativas	11
8.1 GitHub	12
8.2 GitLab	12
8.3 Bitbucket	13
8.4 Azure Repos	13
8.5 Gitea	14
8.6 Forgejo	14
8.7 SourceHut	14

9. Comparação prática das plataformas	14
10. CI/CD aplicado ao versionamento	15
10.1 GitHub Actions	15
10.2 GitLab CI/CD	15
11. Segurança em versionamento	16
11.1 Proteção de branches	16
11.2 Segredos no código	16
11.3 Arquivo .gitignore	17
12. Governança de repositórios em nível sênior	17
12.1 Estrutura mínima de um repositório profissional	17
12.2 CODEOWNERS	18
12.3 Estratégia de permissões	18
13. Boas práticas de code review	18
13.1 O que revisar	19
13.2 Como abrir uma boa PR/MR	19
14. Monorepo vs multirepo	19
14.1 Multirepo	20
14.2 Monorepo	20
15. Versionamento de APIs e contratos	21
16. Git em arquitetura moderna	21
16.1 Git como base para CI/CD	21
16.2 GitOps	21
17. Erros comuns por nível	22
18. Competências esperadas em nível sênior	23
19. Plano de estudo sugerido	23
Fase 1 — Fundamentos	23
Fase 2 — Colaboração	24
Fase 3 — Histórico e recuperação	24
Fase 4 — Automação	25
Fase 5 — Segurança	25
Fase 6 — Nível sênior	26
20. Projeto prático final para consolidar	26
21. Resumo final	27

1. Visão geral

Versionamento de código é a disciplina de registrar, rastrear, comparar, revisar, auditar e integrar mudanças em um projeto de software. Em nível profissional, ele não serve apenas para “salvar histórico”; ele sustenta colaboração, revisão de código, rastreabilidade de requisitos, rollback, release, CI/CD, segurança, governança e auditoria técnica.

Git é o sistema de versionamento distribuído mais usado no desenvolvimento moderno. Em um modelo distribuído, cada desenvolvedor tem uma cópia completa do projeto e do histórico, sem depender o tempo todo de um servidor central. Plataformas como GitHub, GitLab, Bitbucket, Azure Repos, Gitea e Forgejo não substituem o Git; elas adicionam camada de colaboração, pull/merge requests, permissões, automações, segurança, issues, boards, releases, pacotes e CI/CD.

2. Objetivo da trilha

Ao estudar Git e versionamento até nível sênior, o objetivo não é decorar comandos. O objetivo é

desenvolver capacidade de:

- Trabalhar com segurança em bases de código reais.
- Entender histórico, branches, merges, conflitos e releases.
- Colaborar em times com pull requests, code review e políticas de branch.
- Automatizar qualidade com pipelines, testes e verificações obrigatórias.
- Proteger código contra vazamento de segredos, commits indevidos e mudanças sem revisão.
- Definir estratégia de branching e release adequada ao contexto da empresa.
- Administrar repositórios, permissões, governança, monorepos e migrações entre plataformas.

3. Trilha progressiva por nível

Nível básico — Fundamentos essenciais

3.1 O que é controle de versão

Controle de versão é o mecanismo que registra mudanças em arquivos ao longo do tempo. Em projetos de software, isso permite saber quem alterou algo, quando alterou, por que alterou e como voltar ou comparar estados anteriores.

Você deve entender:

- Repositório.
- Arquivo rastreado.
- Commit.
- Histórico.
- Branch.
- Merge.
- Tag.
- Remote.
- Clone.

- Push.
- Pull.
- Fetch.

No básico, o foco é conseguir trabalhar sem destruir histórico, sem sobrescrever alterações de colegas e sem depender de cópias manuais como projeto-final-v2-agora-vai.zip.

3.2 Estados principais do Git

Git trabalha com três áreas fundamentais:

Área	Função
Working Directory	Onde você edita os arquivos localmente.
Staging Area / Index	Área de preparação do próximo commit.
Repository	Banco local de commits, branches, tags e objetos Git.

Fluxo mental:

```
# edita arquivos
git status
# adiciona mudanças ao staging
git add arquivo.java
# cria snapshot versionado
git commit -m "feat: add customer validation"
# envia para o remoto
git push origin minha-branch
```

- O erro comum no início é pensar que git add envia o arquivo para o GitHub. Não envia. Ele apenas prepara a mudança para o commit local.

3.3 Comandos básicos obrigatórios

```
git init
git clone <url>
git status
git add .
git commit -m "mensagem"
git log
git diff
git branch
git switch -c minha-branch
git switch main
git pull
git push
```

Domínio básico significa saber criar uma branch, fazer commits pequenos, enviar para o remoto e abrir uma Pull Request ou Merge Request.

4. Nível intermediário — Trabalho em equipe e fluxo real

4.1 Branches

Uma branch em Git é um ponteiro leve e móvel para um commit. Quando você cria uma branch, o Git cria uma nova referência apontando para um commit existente; conforme novos commits são feitos naquela branch, o ponteiro se move.

Exemplo:

```
git switch -c feature/customer-api
git add .
git commit -m "feat(customer): create customer endpoint"
git push origin feature/customer-api
```

Boas práticas:

- Use nomes claros: feature/, bugfix/, hotfix/, release/.
- Evite branches longas demais.
- Atualize sua branch com frequência.
- Não trabalhe diretamente na main ou master.
- Abra PR/MR pequena e revisável.

4.2 Merge vs Rebase

Git possui duas formas principais de integrar mudanças: merge e rebase. O merge preserva a linha histórica original e cria, quando necessário, um commit de merge. O rebase reaplica commits de uma branch sobre outra base, podendo deixar o histórico mais linear.

Merge

```
git switch main
git pull
git merge feature/customer-api
```

Use quando você quer preservar claramente o histórico de integração.

Rebase

```
git switch feature/customer-api
git fetch origin
git rebase origin/main
```

Use quando você quer atualizar sua branch com a main e manter histórico mais limpo.

Atenção: Regra profissional: não faça rebase de commits que já foram compartilhados com outras pessoas sem alinhamento, porque isso reescreve histórico e pode quebrar o trabalho de outros.

4.3 Conflitos de merge

Conflito acontece quando duas mudanças incompatíveis afetam a mesma região do código ou quando Git não consegue decidir automaticamente qual versão manter.

Fluxo correto:

```
git status
# abrir arquivos com conflito
# resolver manualmente
git add arquivo-com-conflito.java
git rebase --continue
# ou
git commit
```

Erros comuns:

- Aceitar “ours” ou “theirs” sem entender.
- Resolver conflito apagando regra de negócio do colega.
- Resolver conflito sem rodar testes.
- Fazer merge gigante depois de semanas de branch parada.

4.4 Pull Request / Merge Request

Pull Request, no GitHub/Azure/Bitbucket, ou Merge Request, no GitLab, é o processo formal para revisar, discutir, validar e integrar mudanças. No Azure Repos, PRs são usados para revisar código, comentar e aprovar/rejeitar mudanças antes do merge na branch principal.

Uma boa PR/MR deve ter:

- Título claro.
- Descrição do problema.
- Solução aplicada.
- Evidência de teste.
- Prints/logs quando necessário.
- Link para task, issue ou história.
- Checklist de impacto.

Modelo:

```
## Objetivo
• Implementar validação de cliente antes da criação do pedido.
## Alterações
- Adicionado CustomerValidator
- Criado teste unitário para documento inválido
- Ajustado retorno 400 no controller
## Como testar
• 1. Rodar `mvn test`
• 2. Chamar POST /orders com customerId inválido
## Riscos
Baixo. Impacta apenas fluxo de criação de pedido.
```

5. Nível avançado — Git interno, qualidade e automação

5.1 Modelo interno do Git

Git não armazena apenas “diferenças de arquivos” de forma simplista. Internamente, ele usa objetos como blobs, trees e commits. Blobs representam conteúdo de arquivos, trees representam estruturas de diretórios e commits apontam para uma tree, armazenando metadados como autor, data, mensagem e commit pai.

Isso é importante porque explica:

- Por que commits são snapshots.
- Por que branches são baratas.
- Por que hashes mudam quando o conteúdo muda.
- Por que rebase altera hashes.
- Por que o Git consegue verificar integridade.

Um commit é mais do que uma mensagem. Ele é um objeto que aponta para o estado do projeto naquele momento e para seu histórico anterior.

5.2 HEAD, refs e histórico

HEAD representa onde você está no histórico. Normalmente aponta para a branch atual. Em

estado normal:

```
HEAD -> feature/customer-api -> commit abc123
```

Em detached HEAD:

```
HEAD -> commit abc123
```

Detached HEAD ocorre quando você faz checkout direto em um commit ou tag. Isso é útil para inspeção, mas perigoso para trabalho contínuo se você não criar uma branch.

5.3 Reset, revert e restore

Esses comandos confundem muitos devs.

git revert

Cria um novo commit desfazendo outro commit. É seguro para histórico compartilhado.

```
git revert <commit>
```

Use em produção, branches compartilhadas e repositórios corporativos.

git reset

Move o ponteiro da branch. Pode alterar staging e working directory.

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Use com cuidado. --hard pode descartar alterações locais.

git restore

Restaura arquivos no working directory ou staging.

```
git restore arquivo.java
git restore --staged arquivo.java
```

5.4 Stash

git stash guarda mudanças temporariamente sem commit.

```
git stash
git pull
git stash pop
```

Use quando você precisa trocar de branch ou atualizar a base sem perder alterações locais.

Atenção: Erro comum: usar stash como “lixeira permanente” e esquecer mudanças importantes.

5.5 Cherry-pick

Aplica um commit específico em outra branch.

```
git cherry-pick <commit>
```

Usos reais:

- Levar hotfix da main para uma branch de release.
- Aplicar correção específica sem fazer merge completo.
- Recuperar commit útil de uma branch descartada.

Risco: duplicar mudanças e criar conflitos futuros se usado sem controle.

5.6 Bisect

git bisect ajuda a descobrir qual commit introduziu um bug.

```
git bisect start
git bisect bad
git bisect good <commit-antigo>
```

É uma habilidade avançada muito valorizada, porque reduz investigação manual em bases grandes.

5.7 Hooks

Git hooks são scripts executados em eventos específicos do Git. Existem hooks client-side, como pre-commit, e server-side, como hooks relacionados a push. O pre-commit pode validar lint, testes, formatação ou padrões antes de permitir o commit.

Exemplos de uso:

- Bloquear commit com segredo.
- Rodar lint.
- Validar mensagem de commit.
- Impedir commit em branch errada.
- Executar testes rápidos.

Exemplo:

```
.git/hooks/pre-commit
```

- Em times sérios, hooks locais ajudam, mas não substituem validação no servidor ou pipeline, porque hooks locais podem ser ignorados.

6. Convenções profissionais de commits

6.1 Conventional Commits

Conventional Commits define uma convenção leve para mensagens de commit com significado humano e automatizável. A estrutura recomendada é <type>[optional scope]: <description>, com tipos como feat, fix, docs, test, refactor, ci, build e marcação explícita para breaking changes.

Exemplos:

```
feat(order): add order creation endpoint
fix(payment): handle null authorization code
test(customer): cover invalid document scenario
refactor(order): extract validation service
docs(readme): add local setup instructions
ci(pipeline): add integration test stage
```

Para breaking change:

```
feat(api)!: change customer response contract
```

Ou:

```
feat(api): change customer response contract
BREAKING CHANGE: field documentNumber was renamed to taxId
```

6.2 SemVer

Semantic Versioning usa o formato MAJOR.MINOR.PATCH. Correções compatíveis incrementam PATCH, funcionalidades compatíveis incrementam MINOR, e mudanças incompatíveis incrementam MAJOR.

Exemplo:

```
1.2.3
| | └─ patch: correção de bug compatível
| └─── minor: nova funcionalidade compatível
└───── major: mudança incompatível
```

Uso real:

- Bibliotecas.
- APIs públicas.
- SDKs.
- Pacotes npm/Maven.
- Serviços consumidos por outros times.
- Releases versionadas.

7. Estratégias de branching

7.1 GitHub Flow

Modelo simples:

```
main
└─ feature/*
    └─ pull request
        └─ merge na main
```

Indicado para:

- Times pequenos/médios.
- Deploy contínuo.
- Produto web.
- Branches curtas.
- CI forte.

Princípio: a main deve estar sempre pronta para deploy.

7.2 GitFlow

Modelo clássico:

```
main
develop
feature/*
release/*
hotfix/*
```

Indicado para:

- Produtos com releases planejadas.
- Aplicações com ciclos mais longos.
- Ambientes corporativos com homologação formal.

- Softwares versionados instaláveis.

Risco: branches longas, muitos merges e maior custo operacional.

7.3 Trunk-Based Development

Trunk-Based Development defende colaboração em uma única linha principal, normalmente main ou trunk, evitando branches de desenvolvimento longas. O modelo usa branches curtas, integração frequente, feature flags e CI forte para reduzir “merge hell”.

Indicado para:

- Times maduros.
- CI/CD robusto.
- Alta frequência de entrega.
- Microserviços.
- Produtos SaaS.
- Equipes com testes automatizados confiáveis.

Exige:

- Testes rápidos.
- Pipelines confiáveis.
- Feature flags.
- Observabilidade.
- Rollback/fix-forward.
- Revisões pequenas.

7.4 Release branches

Branches de release são usadas quando você precisa estabilizar uma versão enquanto a main continua recebendo evolução.

Exemplo:

```
main
```

```
release/1.8.x hotfix/1.8.1
```

Uso real:

- Aplicativo mobile.
- Sistemas embarcados.
- Software on-premise.
- APIs com contrato formal.
- Sistemas regulados.

8. Plataformas: GitHub, GitLab, Bitbucket, Azure Repos e alternativas

8.1 GitHub

GitHub é forte em colaboração, open source, GitHub Actions, Pull Requests, GitHub Packages, Dependabot, CodeQL, secret scanning, branch protection, environments, releases e integração com Copilot.

Recursos importantes:

- Pull Requests.
- Branch protection.
- Required status checks.
- CODEOWNERS.
- GitHub Actions.
- Issues e Projects.
- Releases.
- Packages.
- Code scanning.
- Secret scanning.
- Dependency review.
- Dependabot.

Em branches protegidas no GitHub, status checks obrigatórios precisam passar antes do merge quando essa regra está habilitada. O GitHub Advanced Security inclui recursos voltados a encontrar vulnerabilidades e vazamentos, como code scanning, secret scanning, push protection, Dependabot e dependency review.

Quando escolher GitHub

- Projetos open source.
- Times que querem grande ecossistema.
- Integração forte com Actions.
- Projetos com foco em comunidade.
- Empresas que usam GitHub Enterprise.
- Times que querem boa experiência de PR.

8.2 GitLab

GitLab é uma plataforma DevSecOps mais integrada, com Git, Merge Requests, CI/CD, container registry, package registry, security scanning, environments, protected branches, approvals, Code Owners e possibilidade forte de self-managed.

Recursos importantes:

- Merge Requests.
- Protected branches.
- Code Owners.
- GitLab CI/CD.
- Runners.

- Environments.
- Security scanning.
- Container Registry.
- Package Registry.
- Approval policies.
- Monorepo pipelines.
- Self-managed enterprise.

No GitLab, Code Owners podem ser usados para exigir aprovação de responsáveis por áreas do código, desde que a branch de destino esteja protegida e a aprovação por Code Owner esteja habilitada.

Branches protegidas no GitLab permitem controlar quem pode fazer merge, push e push direto, além de exigir merge request para mudanças em branches sensíveis.

GitLab CI/CD usa o arquivo `.gitlab-ci.yml`, com jobs, stages, runners e execução automática por eventos como push, merge request ou agendamento.

Quando escolher GitLab

- Empresas que querem DevSecOps mais integrado.
- Ambientes self-hosted.
- Times com CI/CD centralizado no GitLab.
- Organizações com governança forte.
- Projetos que precisam de controle corporativo completo.

8.3 Bitbucket

Bitbucket é a plataforma Git da Atlassian, com integração forte com Jira e Jira Service Management. Também oferece Pull Requests, branch permissions, Bitbucket Pipelines e integração com fluxo de trabalho Atlassian.

Quando escolher Bitbucket

Empresas que usam Jira intensivamente.

Times que querem rastreabilidade issue → branch → PR → deploy.

- Ambientes Atlassian.
- Projetos com gestão de demanda muito ligada a Jira.

8.4 Azure Repos

Azure Repos faz parte do Azure DevOps e oferece Git repos, Pull Requests, branch policies, permissões, integração com Azure Pipelines, Boards, Artifacts e Test Plans.

No Azure Repos, branch policies ajudam a proteger branches importantes, impor padrões de qualidade e exigir Pull Requests para alterações quando políticas obrigatórias estão configuradas.

Quando escolher Azure Repos

- Empresas que usam Azure DevOps.
- Times que já trabalham com Azure Boards/Pipelines.
- Projetos Microsoft/Azure.
- Ambientes corporativos com forte controle de permissão.

- Integração com pipelines YAML do Azure DevOps.

8.5 Gitea

Gitea é uma solução self-hosted de desenvolvimento com Git hosting, code review, colaboração, package registry e CI/CD. É uma alternativa leve para quem quer hospedar sua própria plataforma Git.

Quando escolher Gitea

- Laboratórios internos.
- Times pequenos.
- Ambientes privados.
- Baixo custo operacional.
- Necessidade de self-host simples.
- Projetos onde GitHub/GitLab são pesados demais.

8.6 Forgejo

Forgejo é uma forge Git self-hosted leve, com foco em software livre, privacidade, segurança, federação e controle próprio da infraestrutura.

Quando escolher Forgejo

- Organizações com foco em soberania tecnológica.
- Projetos open source que querem evitar dependência de grandes plataformas.
- Ambientes self-hosted.
- Times que valorizam software livre e controle operacional.

8.7 SourceHut

SourceHut segue uma abordagem mais minimalista e hacker-oriented, com forte apoio a listas de e-mail, revisão por patches e git send-email.

Quando escolher SourceHut

- Projetos open source mais tradicionais.
- Times acostumados com patches por e-mail.
- Desenvolvimento estilo kernel/Linux.
- Ambientes que preferem simplicidade e texto sobre interfaces pesadas.

9. Comparação prática das plataformas

Plataforma	Melhor para	Pontos fortes	Atenção
GitHub	Open source, comunidade, produto moderno	PRs, Actions, Security, Copilot, ecossistema	Recursos avançados podem exigir planos pagos
GitLab	DevSecOps completo	CI/CD nativo, self-host, segurança, governança	Administração pode ser mais complexa
Bitbucket	Times Atlassian	Jira, Pipelines, PRs, branch permissions	Menor ecossistema que GitHub
Azure Repos	Empresas Microsoft/Azure	Azure Boards, Pipelines, políticas corporativas	Menos usado em open source
Gitea	Self-host leve	Simples, leve, Git hosting, CI/CD	Menos recursos enterprise nativos
Forgejo	Software livre/self-host	Privacidade, controle, comunidade livre	Ecossistema menor
SourceHut	Patches e mailing lists	Simples, textual, fluxo open source clássico	Curva cultural diferente para quem vem de PRs

10. CI/CD aplicado ao versionamento

10.1 GitHub Actions

GitHub Actions é a plataforma de CI/CD do GitHub para automatizar build, testes e deploy.

Workflows são definidos em YAML dentro de `.github/workflows`, podem ser acionados por push, pull request, agenda ou execução manual, e são compostos por jobs, steps, actions e runners.

Exemplo Java/Spring:

```
name: Java CI
on:
  pull_request:
    branches: [ "main" ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21
      - name: Run tests
        run: ./mvnw test
```

10.2 GitLab CI/CD

Exemplo equivalente no GitLab:

```
stages:
  - test
  - build
test:
  stage: test
  image: eclipse-temurin:21
  script:
    - ./mvnw test
build:
  stage: build
  image: eclipse-temurin:21
  script:
    - ./mvnw package -DskipTests
```

Um pipeline profissional deve validar:

- Build.
- Testes unitários.
- Testes de integração.
- Lint.
- Análise estática.
- Segurança de dependências.
- Build de imagem Docker.
- Publicação de artefato.
- Deploy em ambiente controlado.

11. Segurança em versionamento

11.1 Proteção de branches

Branches críticas como main, master, develop, release/* e production/* devem ter regras.

Configurações recomendadas:

- Bloquear push direto.
- Exigir PR/MR.
- Exigir aprovação.
- Exigir aprovação de CODEOWNERS.
- Exigir pipeline verde.
- Exigir branch atualizada.
- Bloquear force push.
- Bloquear deleção.
- Exigir commits assinados em contextos sensíveis.
- Exigir scan de segurança.

No GitHub, branch protection pode exigir status checks antes do merge. No GitLab, branches protegidas controlam permissões de merge e push e podem exigir fluxo por Merge Request. No Azure Repos, políticas de branch ajudam a impor padrões de qualidade e mudança.

11.2 Segredos no código

Nunca commite:

```
senha  
token  
API key  
private key  
.env real  
credential.json  
connection string  
certificado privado
```

O GitHub Secret Scanning verifica o histórico Git em todas as branches procurando credenciais hardcoded, como API keys, senhas e tokens.

Boas práticas:

- Usar .env.example, não .env.
- Usar secrets da plataforma.
- Rotacionar credenciais vazadas.
- Remover segredo do histórico quando necessário.
- Adicionar .gitignore.
- Usar secret scanning.
- Usar pre-commit hooks.
- Usar vault ou secret manager.

11.3 Arquivo .gitignore

Exemplo para Java/Spring:

```
target/  
*.class  
*.log  
.env  
.idea/  
.vscode/  
.DS_Store  
*.iml
```

Para Node/Next.js:

```
node_modules/  
.next/  
dist/  
.env  
.env.local  
coverage/
```

Atenção: Erro comum: commitar target/, node_modules/, .env, builds e arquivos gerados.

12. Governança de repositórios em nível sênior

12.1 Estrutura mínima de um repositório profissional

```
.  
├── README.md  
├── CONTRIBUTING.md  
├── CHANGELOG.md  
├── CODEOWNERS  
├── SECURITY.md  
├── LICENSE  
├── .gitignore  
├── .editorconfig  
├── .github/  
│   ├── workflows/  
│   └── pull_request_template.md  
├── docs/  
├── src/  
└── tests/
```

Função de cada arquivo:

Arquivo	Finalidade
README.md	Explica o projeto, setup, execução e contexto.
CONTRIBUTING.md	Define padrão de contribuição.
CHANGELOG.md	Registra mudanças por versão.
CODEOWNERS	Define responsáveis por áreas do código.
SECURITY.md	Define política de reporte de vulnerabilidades.
LICENSE	Define licença de uso.
Pull Request Template	Padroniza revisões.

12.2 CODEOWNERS

Exemplo:

```
# Backend
/src/main/java/br/com/company/orders/ @backend-team
# Infra
/.github/workflows/ @devops-team
/docker/ @devops-team
# Segurança
/security/ @security-team
```

- Uso sênior: CODEOWNERS não é só burocracia. Ele garante que mudanças em áreas críticas sejam revisadas por pessoas com contexto técnico.

12.3 Estratégia de permissões

Modelo recomendado:

Papel	Permissão
Admin/Owner	Configura repositório, permissões e políticas.
Maintainer	Aprova mudanças críticas e releases.
Developer	Cria branches, PRs/MRs e contribui.
Reviewer	Revisa código em áreas específicas.
Read-only	Consulta código e documentação.

Regras:

- Poucas pessoas com admin.
- Ninguém faz push direto em produção.
- Service accounts separadas de contas humanas.
- Tokens com menor privilégio possível.
- Revisão obrigatória para arquivos críticos.
- Auditoria periódica de acesso.

13. Boas práticas de code review

13.1 O que revisar

Em nível sênior, code review não é procurar “espaço em branco”. É validar qualidade, risco e manutenção.

Checklist:

- A solução resolve o problema correto?
- Há teste cobrindo regra de negócio?
- O código é legível?
- A alteração quebra contrato de API?
- Há impacto em performance?
- Há impacto em segurança?
- Há risco de concorrência?
- Há alteração de schema?
- Há rollback possível?
- Logs expõem dados sensíveis?
- A PR está pequena o suficiente?
- O pipeline passou?

13.2 Como abrir uma boa PR/MR

Ruim:

ajustes

Bom:

```
feat(order): add validation before order creation
```

Descrição boa:

```
## Contexto
```

- Pedidos estavam sendo criados sem validar documento do cliente.

```
## Solução
```

Adicionado CustomerDocumentValidator antes da criação do pedido.

```
## Testes
```

- CustomerDocumentValidatorTest
- OrderServiceTest
- mvn test executado localmente

```
## Impacto
```

Fluxo POST /orders passa a retornar 400 para documento inválido.

14. Monorepo vs multirepo

14.1 Multirepo

Cada serviço/projeto tem seu próprio repositório.

Vantagens:

- Isolamento.
- Permissões por serviço.
- Pipelines menores.
- Release independente.
- Menor complexidade inicial.

Desvantagens:

- Difícil padronizar.
- Duplicação de configuração.
- Mudanças cross-service exigem várias PRs.
- Gestão de dependências mais complexa.

14.2 Monorepo

Vários projetos dentro de um único repositório.

Vantagens:

- Visão única.
- Refatorações transversais.
- Padronização.
- Melhor compartilhamento de libs.
- Histórico centralizado.

Desvantagens:

- Pipelines mais complexos.
- Permissões mais difíceis.
- Build pode ficar pesado.
- Exige tooling forte.
- Exige estratégia de path filters, owners e cache.

Estrutura exemplo:

```
.
├── apps/
│   ├── web/
│   └── admin/
├── services/
│   ├── ms-auth/
│   ├── ms-orders/
│   └── ms-payments/
├── packages/
│   ├── ui/
│   └── shared/
└── infra/
```

Em nível sênior, a decisão entre monorepo e multirepo deve considerar autonomia dos times, acoplamento entre serviços, governança, CI/CD, segurança, dependências e frequência de mudança.

15. Versionamento de APIs e contratos

Git não versiona apenas código; ele suporta a governança de contratos.

Você deve versionar:

- Código fonte.
- Migrations.
- OpenAPI/Swagger.
- AsyncAPI.
- Scripts de infraestrutura.
- Dockerfiles.
- Helm charts.
- Terraform.
- Documentação técnica.
- ADRs.
- Testes de contrato.

Estrutura recomendada:

```
docs/  
├── adr/  
│   ├── 0001-use-postgresql.md  
│   └── 0002-use-kafka-for-events.md  
contracts/  
├── openapi.yaml  
└── asyncapi.yaml
```

16. Git em arquitetura moderna

16.1 Git como base para CI/CD

Cada push ou PR pode acionar:

```
commit → pipeline → testes → build → scan → artefato → deploy
```

Em times maduros, o repositório vira o ponto de entrada da engenharia:

- Código.
- Infraestrutura.
- Documentação.
- Políticas.
- Workflows.
- Releases.
- Evidência de auditoria.

16.2 GitOps

GitOps usa Git como fonte de verdade para estado desejado de infraestrutura e aplicações.

Exemplo:

```
Alteração em manifest Kubernetes
↓
Pull Request
↓
Review + pipeline
↓
Merge na main
↓
Argo CD / Flux aplica no cluster
```

GitOps exige disciplina forte:

- Branch protection.
- PR obrigatória.
- Auditoria.
- Rollback via commit.
- Separação por ambientes.
- Segredos fora do Git.
- Revisão de infraestrutura como código.

17. Erros comuns por nível

Iniciante

- Fazer tudo na main.
- Não usar git status.
- Commits gigantes.
- Mensagens ruins.
- Subir .env.
- Confundir pull com fetch.
- Resolver conflito sem testar.

Intermediário

- Rebase incorreto em branch compartilhada.
- PR grande demais.
- Fazer force push sem cuidado.
- Não atualizar branch.
- Usar merge sem entender histórico.
- Não usar tags/releases.

Avançado

- Pipeline lento e sem cache.
- Ausência de branch protection.
- CODEOWNERS mal configurado.
- Falta de política de secrets.
- Monorepo sem estratégia.

- Releases manuais sem rastreabilidade.

Sênior

- Escolher GitFlow para time que precisa de deploy contínuo.
- Escolher trunk-based sem testes suficientes.
- Permissões excessivas.
- Falta de rollback.
- Falta de auditoria.
- Não versionar contratos.
- Não definir estratégia para breaking changes.
- Não medir saúde do fluxo de desenvolvimento.

18. Competências esperadas em nível sênior

Um profissional sênior deve saber:

- Explicar o modelo interno do Git.
- Escolher entre merge, rebase, squash, cherry-pick e revert.
- Definir estratégia de branches por contexto.
- Configurar branch protection.
- Estruturar Pull Request templates.
- Definir CODEOWNERS.
- Criar padrões de commits.
- Aplicar SemVer.
- Automatizar changelog e release.
- Configurar CI/CD.
- Integrar testes, lint e scans.
- Controlar permissões.
- Tratar vazamento de segredo.
- Migrar repositórios entre plataformas.
- Estruturar monorepo ou multirepo.
- Definir governança de repositórios.
- Melhorar tempo de review e lead time.
- Ensinar o time a usar Git corretamente.

19. Plano de estudo sugerido

Fase 1 — Fundamentos

Estude:

- O que é Git.
- Repositório local.

- Commit.
- Branch.
- Remote.
- Push/pull/fetch.
- .gitignore.
- GitHub básico.

Pratique:

```
git init
git add
git commit
git log
git branch
git switch
git merge
git remote
git push
```

- Entrega prática: criar um repositório com README, branches e commits organizados.

Fase 2 — Colaboração**Estude:**

- Pull Request.
- Merge Request.
- Code review.
- Conflitos.
- Rebase.
- Squash merge.
- Templates.
- Issues.

Pratique:

- Criar PR.
- Revisar PR.
- Simular conflito.
- Resolver conflito.
- Usar rebase.
- Usar squash.

Entrega prática: projeto com duas branches e PR documentada.

Fase 3 — Histórico e recuperação**Estude:**

- Reset.
- Revert.
- Restore.

- Stash.
- Reflog.
- Cherry-pick.
- Bisect.
- Tags.

Pratique:

```
git revert  
git reset  
git restore  
git stash  
git reflog  
git cherry-pick  
git bisect  
git tag
```

Entrega prática: simular bug introduzido e encontrar com git bisect.

Fase 4 — Automação**Estude:**

- GitHub Actions.
- GitLab CI/CD.
- Azure Pipelines ou Bitbucket Pipelines.
- Checks obrigatórios.
- Testes automatizados.
- Build.
- Deploy.
- Cache.
- Matrix builds.

Entrega prática: pipeline rodando testes em PR.

Fase 5 — Segurança**Estude:**

- Secret scanning.
- Code scanning.
- Dependabot.
- Branch protection.
- CODEOWNERS.
- Signed commits.
- Permissões.
- Tokens.
- .env seguro.

Entrega prática: repositório com branch protegida, PR obrigatória, pipeline obrigatório e CODEOWNERS.

Fase 6 — Nível sênior

Estude:

- Estratégia de branching.
- Trunk-Based Development.
- GitFlow.
- Release management.
- SemVer.
- Conventional Commits.
- Monorepo.
- GitOps.
- Governança.
- Auditoria.
- Migração de plataformas.

Entrega prática: definir uma política completa de versionamento para um projeto real.

20. Projeto prático final para consolidar

Crie um projeto Java/Spring ou Next.js com:

```
.
├── README.md
├── CONTRIBUTING.md
├── CHANGELOG.md
├── CODEOWNERS
├── SECURITY.md
├── .gitignore
├── .github/workflows/ci.yml
├── docs/adr/
└── src/
```

Requisitos:

- Branch main protegida.
- PR obrigatória.
- Pelo menos 1 aprovação.
- Pipeline obrigatório.
- Testes automatizados.
- Conventional Commits.
- Tag de release.
- Changelog.
- CODEOWNERS.
- .env.example.
- Sem segredos no repositório.
- Documentação de fluxo Git.

21. Resumo final

- Para nível básico, Git é ferramenta de histórico.
- Para nível intermediário, Git é ferramenta de colaboração.
- Para nível avançado, Git é ferramenta de automação, qualidade e segurança.
- Para nível sênior, Git é parte da arquitetura de entrega de software.

O profissional sênior não apenas usa git commit e git push; ele define como o time versiona, revisa, protege, integra, entrega, audita e recupera software em produção.